

M3 Project 1: Sorting Algorithms - Performance Analysis Report

Name: Vincent Goldberg

Course: C343 Data Structures

Date: June 12, 2026

Testing Results (Task 2)

All five algorithms were tested on two random 10-element lists. Each screenshot shows the original unsorted list followed by the output of each algorithm, with a PASS/FAIL check that verifies the result is in ascending order.

```
===== Test List 1 =====
Original array:
[1368455831, 592571611, 2086135028, 1115859150, 32092169, 763831799, 660969426, 1880133619, 1596610151, 1453580274]

After Insertion Sort:
[32092169, 592571611, 660969426, 763831799, 1115859150, 1368455831, 1453580274, 1596610151, 1880133619, 2086135028]
PASS
After Bubble Sort:
[32092169, 592571611, 660969426, 763831799, 1115859150, 1368455831, 1453580274, 1596610151, 1880133619, 2086135028]
PASS
After Selection Sort:
[32092169, 592571611, 660969426, 763831799, 1115859150, 1368455831, 1453580274, 1596610151, 1880133619, 2086135028]
PASS
After Merge Sort:
[32092169, 592571611, 660969426, 763831799, 1115859150, 1368455831, 1453580274, 1596610151, 1880133619, 2086135028]
PASS
After Quick Sort:
[32092169, 592571611, 660969426, 763831799, 1115859150, 1368455831, 1453580274, 1596610151, 1880133619, 2086135028]
PASS
```

```
===== Test List 2 =====
Original array:
[729231488, 403946837, 1364148784, 1710104310, 500748648, 1349089038, 352793353, 2043355966, 1616559991, 1895698730]

After Insertion Sort:
[352793353, 403946837, 500748648, 729231488, 1349089038, 1364148784, 1616559991, 1710104310, 1895698730, 2043355966]
PASS
After Bubble Sort:
[352793353, 403946837, 500748648, 729231488, 1349089038, 1364148784, 1616559991, 1710104310, 1895698730, 2043355966]
PASS
After Selection Sort:
[352793353, 403946837, 500748648, 729231488, 1349089038, 1364148784, 1616559991, 1710104310, 1895698730, 2043355966]
PASS
After Merge Sort:
[352793353, 403946837, 500748648, 729231488, 1349089038, 1364148784, 1616559991, 1710104310, 1895698730, 2043355966]
PASS
After Quick Sort:
[352793353, 403946837, 500748648, 729231488, 1349089038, 1364148784, 1616559991, 1710104310, 1895698730, 2043355966]
PASS
```

Edge cases were also tested: an empty array, a single-element array, and an array with duplicate values.

===== Edge Case: Empty Array =====

Original array:

[]

After Insertion Sort:

[]

PASS

After Bubble Sort:

[]

PASS

After Selection Sort:

[]

PASS

After Merge Sort:

[]

PASS

After Quick Sort:

[]

PASS

===== Edge Case: Single Element =====

Original array:

[42]

After Insertion Sort:

[42]

PASS

After Bubble Sort:

[42]

PASS

After Selection Sort:

[42]

PASS

After Merge Sort:

[42]

PASS

After Quick Sort:

[42]

PASS

```

===== Edge Case: Duplicates =====
Original array:
[7, 3, 7, 3, 7, 3, 7]

After Insertion Sort:
[3, 3, 3, 7, 7, 7, 7]
PASS
After Bubble Sort:
[3, 3, 3, 7, 7, 7, 7]
PASS
After Selection Sort:
[3, 3, 3, 7, 7, 7, 7]
PASS
After Merge Sort:
[3, 3, 3, 7, 7, 7, 7]
PASS
After Quick Sort:
[3, 3, 3, 7, 7, 7, 7]
PASS

```

Test Setup

Each algorithm was timed on randomly generated int arrays (values up to Integer.MAX_VALUE) at five sizes: 1,000 / 10,000 / 100,000 / 1,000,000 / 10,000,000. Every algorithm/size combination was run 5 times, the slowest run was discarded as an outlier, and the remaining 4 runs were averaged. Only the sort itself was timed (data generation excluded). Any run that went past 20 minutes was cut off and recorded as "> 20 m". Once an algorithm timed out at one size, the larger sizes were skipped and also recorded as "> 20 m", since these algorithms only get slower as n grows.

During the evaluation, console output was used to show live progress with each dot representing a completed sort, an X representing a timeout, and -- indicating a skipped dataset. The results were printed as a table to the console as a sanity check.

```

===== Performance Evaluation =====
Trials per cell: 5 (discarding slowest)

Insertion:      .   .   .   .   X
Bubble:         .   .   .   X  --
Selection:      .   .   .   .   X
Merge:          .   .   .   .   .
Quick:          .   .   .   .   .

```

```

===== Results Table =====
Algorithm      1K      10K     100K     1M      10M
-----
Insertion      < 1 ms   6 ms    374 ms   38.47 s  > 20 m
Bubble         1 ms    34 ms    7.43 s   > 20 m   > 20 m
Selection      1 ms    28 ms    3.18 s   1094.86 s > 20 m
Merge          < 1 ms   1 ms     10 ms    113 ms   1.29 s
Quick          < 1 ms   1 ms     6 ms     77 ms    859 ms

```

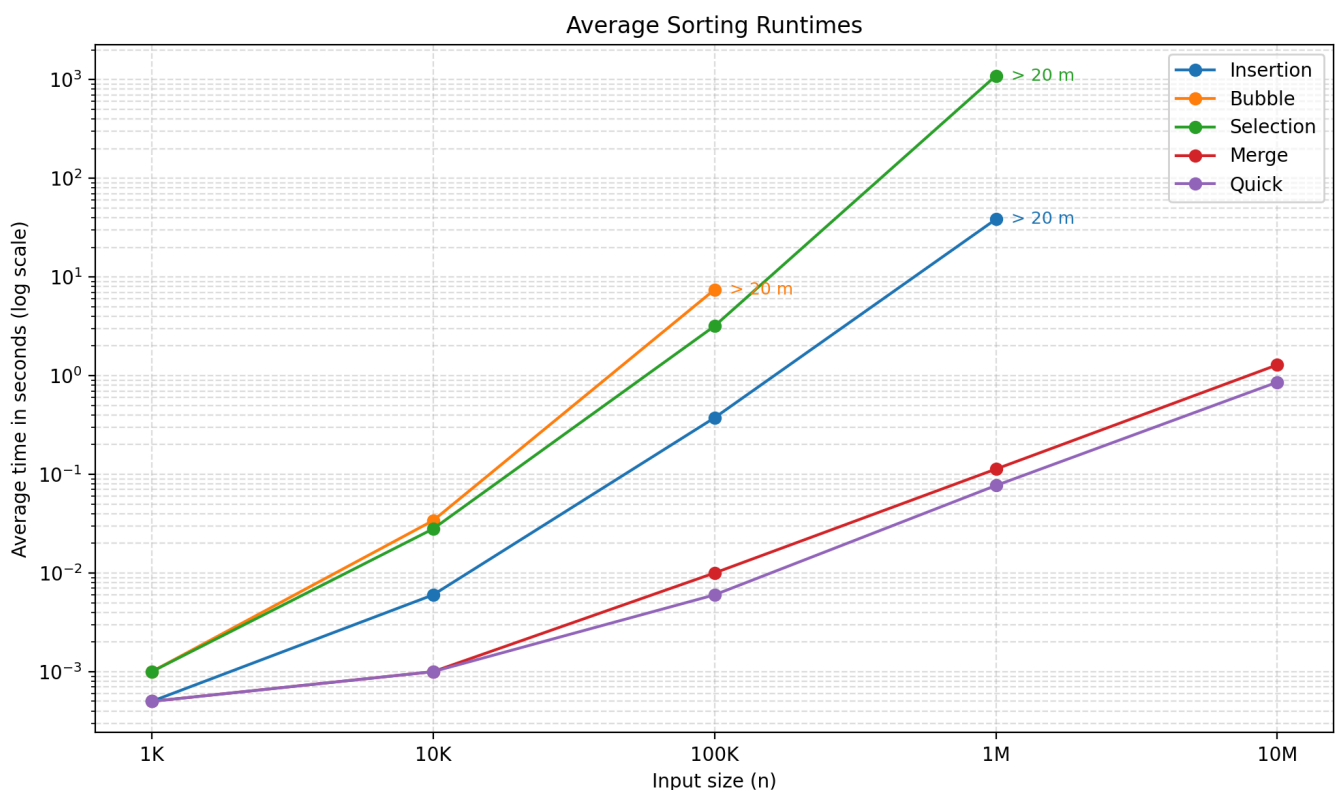
Results Table

The average times of the evaluations were saved to a `performance_results.csv`. This CSV was utilized in a Python script to create visuals that represented the results of the sorting algorithms.

Average Sorting Runtimes

	1,000	10,000	100,000	1,000,000	10,000,000
Insertion	< 1 ms	6 ms	374 ms	38.47 s	> 20 m
Bubble	1 ms	34 ms	7.43 s	> 20 m	> 20 m
Selection	1 ms	28 ms	3.18 s	1094.86 s	> 20 m
Merge	< 1 ms	1 ms	10 ms	113 ms	1.29 s
Quick	< 1 ms	1 ms	6 ms	77 ms	859 ms

Chart



Both axes use a log scale so the fast and slow algorithms fit on one chart. Some lines end where the algorithm exceeded the 20 minute limit at the next size. Times that measured under 1 ms are plotted at 0.5 ms since a log scale can't show 0.

Observations and Conclusions

The results split the five algorithms into two clear groups: the $O(n^2)$ sorts (insertion, bubble, selection) and the $O(n \log n)$ sorts (merge, quick). On the log chart this shows up as two different slopes. Every time n grows 10x, the quadratic sorts take roughly 100x longer (insertion went from 374 ms at 100K to 38.47 s at 1M, about a 103x jump), while merge and quick only take around 10x longer. That difference is what makes the quadratic group fall off a cliff; bubble sort couldn't finish 1,000,000 numbers inside 20

minutes, and selection sort just barely made it at approximately 18.2 minutes. Meanwhile quick sort handled 10,000,000 numbers in under a second.

Below about 10,000 elements, the algorithm choice was less important. All five finished in 34 ms or less, so for small inputs a simple sort is perfectly fine. But as the dataset size increases, algorithm selection becomes important. Therefore, for any dataset that could grow large, an $O(n \log n)$ sort is the only reasonable option, because the quadratic sorts go from "fine" to "won't finish today" within two orders of magnitude. Quick sort is the best general-purpose pick here, and should be the algorithm of choice for large datasets. Merge sort is worth the small extra cost when you need a stable sort or a guaranteed $O(n \log n)$ worst case. Insertion sort still has a niche for very small or nearly-sorted inputs. Bubble sort and selection sort seem to mostly serve as teaching examples; nothing in these results gives a reason to use either in practice.